



Học viện Công nghệ Bưu chính Viễn thông
Khoa Công nghệ Thông tin 1

Nhập môn học sâu

Đại số tuyến tính

TS. NGUYỄN XUÂN ĐỨC

Nội dung

- ❑ Vector, ma trận
- ❑ Trị riêng, vector riêng
- ❑ Singular value decomposition
- ❑ Ứng dụng đại số trong học sâu

Vector, ma trận

Scalar

□ ĐN: một giá trị duy nhất là số nguyên (số thực).

□ Ký hiệu: x , y , z , ...

`x = torch.tensor(3.0)`

`y = torch.tensor(2.0)`

`x + y, x * y, x / y, x ** y`

Vector

□ ĐN: một mảng các số với độ dài cố định

`x = torch.arange(3)`

□ Trong toán học, vector được viết dưới dạng: $x = \begin{bmatrix} x_1 \\ \dots \\ x_n \end{bmatrix}$

Ma trận

□ ĐN: tensor bậc 2

```
A = torch.arange(6).reshape(3, 2)
```

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

Ma trận chuyển vị

☐ Ma trận chuyển vị: $A.T$

☐ Ma trận đối xứng: $A == A.T$

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

Tensor

☐ ĐN: mảng bậc n

`torch.arange(24).reshape(2, 3, 4)`

```
tensor([[[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]],
        [[12, 13, 14, 15],
         [16, 17, 18, 19],
         [20, 21, 22, 23]]])
```


Kiểu của tensor

❑ Tensor chỉ chứa dữ liệu kiểu số và kiểu bool (True/False)

❑ Các kiểu dữ liệu phổ biến:

- torch.float16
- torch.float32
- torch.float64
- torch.int8
- torch.int16
- torch.int32
- torch.int64
- torch.bool

Tính chất phép toán trên ma trận

```
A = torch.arange(6, dtype=torch.float32).reshape(2, 3)
```

```
B = A.clone() # cấp phát bộ nhớ cho copy của ma trận
```

```
A, A + B
```

```
(tensor([[0., 1., 2.],  
         [3., 4., 5.]]),  
 tensor([[ 0.,  2.,  4.],  
         [ 6.,  8., 10.])))
```

Phép nhân Hadamard

□ ĐN: phép nhân tương ứng từng phần tử của hai ma trận gọi là phép nhân Hadamard.

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & \dots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \dots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \dots & a_{mn}b_{mn} \end{bmatrix}$$

Phép toán giữa ma trận và scalar

`a = 2`

`X = torch.arange(24).reshape(2, 3, 4)`

`a + X, (a * X).shape`

```
(tensor([[[ 2,  3,  4,  5],
          [ 6,  7,  8,  9],
          [10, 11, 12, 13]],
         [[14, 15, 16, 17],
          [18, 19, 20, 21],
          [22, 23, 24, 25]]]),
 torch.Size([2, 3, 4]))
```

Reduction

- ❑ Tính tổng các phần tử của tensor

```
x = torch.arange(3, dtype=torch.float32)
```

```
x, x.sum()
```

```
(tensor([0., 1., 2.]), tensor(3.))
```

- ❑ Tổng theo một chiều nhất định

```
A.shape, A.sum(axis=0).shape
```

```
(torch.Size([2, 3]), torch.Size([3]))
```

- ❑ Tổng theo tất cả các chiều **tương đương** tính tổng tất cả các phần tử

```
A.sum(axis=[0, 1]) == A.sum() # tương đương A.sum()
```

```
tensor(True)
```

Tính tổng giữ nguyên số chiều

- ❑ Sử dụng tham số `keepdims=True` để giữ nguyên số chiều của tensor

```
sum_A = A.sum(axis=1, keepdims=True)
```

```
sum_A, sum_A.shape
```

```
(tensor([[ 3.],
```

```
        [12.])),
```

```
torch.Size([2, 1]))
```

- ❑ Chia A cho `sum_A` để tạo thành ma trận có tổng mỗi hàng bằng 1.

```
A / sum_A
```

```
tensor([[0.0000, 0.3333, 0.6667],
```

```
        [0.2500, 0.3333, 0.4167]])
```

Tích vô hướng

❑ Tích vô hướng của 2 vector: $x^T y = \sum_{i=1}^d x_i y_i$

```
y = torch.ones(3, dtype = torch.float32)
```

```
x, y, torch.dot(x, y)
```

```
(tensor([0., 1., 2.]), tensor([1., 1., 1.]), tensor(3.))
```

❑ Tích vô hướng của hai vector bằng tổng tích element-wise:

```
torch.sum(x * y)
```

Nhân ma trận - vector

□ A.shape, x.shape, torch.mv(A, x), A@x

$$\mathbf{Ax} = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_m^\top \end{bmatrix} \mathbf{x} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{x} \\ \mathbf{a}_2^\top \mathbf{x} \\ \vdots \\ \mathbf{a}_m^\top \mathbf{x} \end{bmatrix}$$

Nhân ma trận

- Gọi a_i^T là vector hàng biểu diễn hàng thứ i của ma trận A và b_j là ma trận cột biểu diễn cột thứ j của ma trận B
- Ma trận $C=AB$ có từng phần tử c_{ij} được tính bằng tích vô hướng của a_i^T và b_j :

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1k} \\ a_{21} & a_{22} & \cdots & a_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nk} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1m} \\ b_{21} & b_{22} & \cdots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{k1} & b_{k2} & \cdots & b_{km} \end{bmatrix}$$

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} \mathbf{a}_1^T \\ \mathbf{a}_2^T \\ \vdots \\ \mathbf{a}_n^T \end{bmatrix} [\mathbf{b}_1 \quad \mathbf{b}_2 \quad \cdots \quad \mathbf{b}_m] = \begin{bmatrix} \mathbf{a}_1^T \mathbf{b}_1 & \mathbf{a}_1^T \mathbf{b}_2 & \cdots & \mathbf{a}_1^T \mathbf{b}_m \\ \mathbf{a}_2^T \mathbf{b}_1 & \mathbf{a}_2^T \mathbf{b}_2 & \cdots & \mathbf{a}_2^T \mathbf{b}_m \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{a}_n^T \mathbf{b}_1 & \mathbf{a}_n^T \mathbf{b}_2 & \cdots & \mathbf{a}_n^T \mathbf{b}_m \end{bmatrix}$$

Chuẩn (norm)

□ Chuẩn (norm) là hàm ánh xạ vector sang scalar thỏa mãn các điều kiện:

- $\|\alpha x\| = |\alpha| \|x\|$
- $\|x + y\| \leq \|x\| + \|y\|$
- $\|x\| > 0$ với mọi x khác 0

□ Chuẩn l1: $\|x\|_1 = \sum_{i=1}^n |x_i|$

□ Chuẩn l2: $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$

□ Chuẩn l_p : $\|x\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$

Phân tích giá trị riêng

Trị riêng, vector riêng

❑ Ma trận $A: \begin{pmatrix} 2 & 0 \\ 0 & -1 \end{pmatrix}$

❑ Nhân A với vector $v = [x, y]^T$, ta được $Av = [2x, -y]^T$

❑ Tồn tại vector v sao cho góc không thay đổi nhưng chỉ thay đổi về độ lớn, gọi là vector riêng:
 $Av = \lambda v$, λ gọi là trị riêng

❑ Giải trị riêng: tìm λ sao cho $\det(A - \lambda I) = 0$

Phân tích ma trận

□ $A = \begin{bmatrix} 2 & 1 \\ 2 & 3 \end{bmatrix}$

□ Gọi $W = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix}$ là ma trận có các cột là vector riêng của A.

□ Gọi $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}$ là ma trận với các trị riêng ở trên đường chéo

□ Từ định nghĩa, ta có $AW = W\Sigma$.

□ Ma trận W khả đảo, nên nếu ta nhân W^{-1} vào cả hai vế, ta có: $A = W\Sigma W^{-1}$

Singular Value Decomposition

Phát biểu

□ Ma trận $A_{m \times n}$ có thể phân tích thành dạng: $A = U\Sigma V^T$

➤ Trong đó, U , V là các ma trận trực giao, Σ là ma trận **đường chéo không vuông**

$$\begin{array}{c} \boxed{A_{m \times n}} = \boxed{U_{m \times m}} \times \boxed{\Sigma_{m \times n}} \times \boxed{V_{n \times n}^T} \\ (m < n) \end{array}$$

$$\begin{array}{c} \boxed{A_{m \times n}} = \boxed{U_{m \times m}} \times \boxed{\Sigma_{m \times n}} \times \boxed{V_{n \times n}^T} \\ (m > n) \end{array}$$

Liên hệ với phân tích trị riêng

□ Biến đổi:

$$\begin{aligned} AA^T &= U\Sigma V^T (U\Sigma V^T)^T \\ &= U\Sigma V^T V \Sigma^T U^T \\ &= U\Sigma \Sigma^T U^T = U\Sigma \Sigma^T U^{-1} \end{aligned}$$

trong đó $V^T V = I$ do V là ma trận trực giao.

Liên hệ với PCA

- Gọi $X_{n \times p}$ là ma trận dữ liệu, giả sử các cột được chuẩn hóa giá trị trung bình về 0 (mean-centered)
- PCA tìm vector riêng của ma trận hiệp phương sai: $\Sigma = \frac{1}{n-1} X^T X$ (kích thước $p \times p$)
- Áp dụng SVD cho X : $X = USV^T$
- Khi đó, ma trận hiệp phương sai: $\Sigma = \frac{1}{n-1} X^T X = V \frac{S^2}{n-1} V^T$

Ứng dụng đại số trong học sâu

- ❑ Biểu diễn dữ liệu: tensor
- ❑ Nhân ma trận: dùng trong mọi tầng nơ ron, là cơ sở để tối ưu tốc độ dùng GPU
- ❑ PCA: tiền xử lý dữ liệu, giảm chiều
- ❑ Chuẩn (norm): sử dụng trong giảm quá khớp (overfit) của thuật toán